High-Pop Server Tuning Guide

Convars, Facepunch's 2026 engine changes and plugin performance for PulseRust

Researched 5 November 2026 (force-wipe day) • Covers Rust builds through the October 2026 Livestock update plus Facepunch's commit log up to 4 November

How this was researched

Most Rust "performance convar" lists online come from hosting-company blogs. Many repeat settings that no longer exist, never applied to servers, or do something different from what is claimed. Every recommendation here was checked against the server's own code or Facepunch's own words:

- Decompiled Rust server source (late-2023 and August-2024 builds) to see what each convar actually controls.
- The April 2026 server command list (1,651 commands generated from the game's own ConsoleGen table) and a 2025-26 convar value dump, for current names and defaults.
- Facepunch devblogs, changelists and the main-branch commit log for April to November 2026. rust.facepunch.com could not be reached from the research environment, so these were read through search extracts and secondary coverage. Items marked (reported) are not confirmed against the primary text.
- Carbon and Oxide framework source for the profiling tools, and a public snapshot of 1,561 uMod plugins.

Values marked **TEST** change behaviour in ways that need a staging check first.

Executive Summary

On a 200+ player Rust server in late 2026, four things move server FPS: **(1) entity count**, **(2) plugin main-thread time and garbage**, **(3) Facepunch's new multithreaded player-update "Jobs" modes** and **(4) the CPU**. Of PulseRust's existing changes, reduced animal populations help a little. Reduced tick rates do essentially nothing, and lowering player tick rates below 16 is silently undone by the game. Reduced decay can actively hurt if it means bases decay more slowly.

The biggest wins available right now cost nothing. Let Jobs 4 run: Facepunch's commit log shows it becoming the default with today's patch. Stop starving the process of CPU cores, keep decay at full speed, and remove the plugin disk-I/O stalls already identified in the Kits, PulseDailyGems, UltimateLeaderboard and ClanTree audits.

Top 12 actions, in priority order

#	Action	Why (evidence)	Effort / risk
1	After today's patch, confirm `server.useplayerupdatejobs` reads **4**. Remove any 0/1/2/3 overrides.	Jobs 4 parallelises player updates and entity snapshot serialisation. In Facepunch's synthetic test, 10,000 entities went from 64.4 ms to 10.6 ms. Merged as the default on 3 Nov.	5 min / low
2	Give the Rust process a full CCD of fast cores. Never pin it to 1-2 cores.	Jobs 3/4, navigation and IK now run on worker threads. A 13 Oct Facepunch commit makes job threads auto-size to the CPU affinity mask.	15 min / low
3	Keep `decay.scale 1` and upkeep on. Remove abandoned bases with a cleanup plugin.	Entity count drives late-wipe cost: saves, snapshots, memory. Decay CPU is fixed at ~10 entities per frame whatever the settings.	30 min / low-med
4	Set `fps.limit` back to 240, or at least 100 on a shared host.	Many systems run on per-frame ms budgets, so fewer frames means less AI, turret, IO and decay work per second.	1 min / low
5	Set `server.tickrate` back to the default 10.	It only paces RCON, Rust+, Nexus, auth time-outs and the OnTick plugin hook. It is not the simulation rate.	1 min / none
6	Keep animals at ~50% of default. Don't cut further.	Facepunch caps animal AI per frame: more animals means slower reactions, not lower FPS. Animals are a tiny share of entities.	done / none
7	Fix critical plugin findings K-01, DG-01, UL-01..03 and CT-01..03. Stagger OnServerSave writes.	Plugin disk I/O lands in the same frames as Rust's own single-frame save write.	hours / low

#	Action	Why (evidence)	Effort / risk
8	Profile every wipe: Carbon `c.profile` and `c.hookinfo`, plus Facepunch `profile.perfsnapshot` at peak.	Only measurement shows which hook or system owns the slow frames.	30 min per wipe / none
9	Delete dead convars (`gc.buffer`, `physics.steps`, legacy entries; see Section 6).	They do nothing on a 2026 server and hide the real settings.	10 min / none
10	Shorten lifetimes of abandoned vehicles and dropped items modestly.	Bikes and snowmobiles last 24 h outside by default, and loose items are physics bodies.	10 min / gameplay
11	Keep `server.saveinterval 600`. Move to 900 only if the save log shows write+disk over ~0.5 s.	Rust writes the whole save buffer in one frame, then writes to disk synchronously.	5 min / low
12	On staging, trial `-useNewNavmesh` and the navmesh carve options.	Background build, disk cache and multithreaded pathing. NPC plugins must be checked against it.	1-2 h / medium

1. Where a Rust server's frame actually goes

`ServerMgr.Update()` runs once per server frame on the main thread. In the August 2024 server source it calls, in order: EAC and platform updates, network receive, then the building manager (stability queue, surroundings queue, skin queue and the decay round-robin), then every player's server update. After that come the budgeted turret, gun-trap and dropped-item queues, the connection queue, the electricity (IO) queue and the AI think queues. Human and animal queues alternate frames while `ai.spliceupdates` is on. Pets, growables, life-story stats, junk-pile water, industrial transfers and AntiHack run last. Saves, population refills and spawns run as coroutines that yield between frames.

Four consequences shape every recommendation in this guide:

- **Most systems are capped by a per-frame millisecond budget.** When the server is busy, they don't take more frame time; they fall behind instead. Turrets acquire targets later, electricity lags and animals react slower. Raising a budget never raises FPS. Lowering one protects FPS at that system's expense.
- **Server FPS sets the throughput of those budgets.** A server held at 30 FPS does half the AI, turret and IO work per second of one at 60 FPS. Artificially low `fps.limit` values hurt for this reason.
- **Entity count is the cost that grows all wipe.** Every entity is serialised on every save, snapshotted to each player entering its network group and held in memory. Physical ones are also simulated.
- **Plugin hooks run inline on the main thread** inside these same frames, with no budget of their own.

Per-frame budgets in the current build

System	Budget convar	2026 value	If you lower it
Animal AI think	`aithinkmanager.animalframebudgetms`	2 ms	Animals react slower
NPC / human AI think	`aithinkmanager.framebudgetms`	2 ms	Scientists react slower
HTN agents (animal / player / junkpile)	`aimanager.ai_htn_*_tick_budget`	4 ms each	Agents tick less often
Async pathfinding	`aimanager.pathfindingiterationsperframe`	100 nodes	Paths resolve slower (Facepunch tuning range 50-500)
Auto-turret target scan	`autoturret.scan_budget_ms`	0.5 ms	Turrets acquire later
Shotgun traps	`guntrap.gun_trap_budget_ms`	0.5 ms	Traps react later
Electricity (high / low priority)	`ioentity.framebudgetelectrichighpriorityms` / `...lowpriorityms`	1 / 0.5 ms	Circuits update later
Fluid / kinetic / generic / industrial IO	`ioentity.framebudget{fluid,kinetic,generic,industrial}ms`	0.25 / 1 / 1 / 0.25 ms	Pumps and conveyors lag
Industrial transfers	`server.industrialframebudgetms`	0.5 ms	Conveyors move less per second
Planters / growables	`growableentity.framebudgetms`	0.25 ms	Plants update later
Ovens / food spoiling	`server.ovencookbudgetms` / `server.foodspoilingbudgetms`	0.25 / 0.05 ms	Smelting / spoil ticks lag
Save-cache warm-up	`server.saveframebudget`	5 ms	Saves take longer wall-time

System	Budget convar	2026 value	If you lower it
Stability checks	`stability.stabilityqueue` / `stability.surroundingsqueue`	9 / 3 ms	Collapses resolve later
Building + world decay	hard-coded	5 + 5 entities per frame	n/a (see Section 2.3)

Values are from the 2025-26 convar dump and April 2026 help text; the frame order and decay constants are from the August 2024 server source.

2. Review of what PulseRust has already changed

You mentioned three changes: reduced animal populations, reduced tick rates and reduced decay rates. Here is what each one actually does in the server code.

2.1 KEEP - SMALL GAIN Reduced animal populations

What the code does. Each `<animal>.population` convar is a density in animals per km². The population's target count is map area × density × a population scale. That scale is lerped between `spawn.min_density` and `spawn.max_density` by online players ÷ `server.maxplayers`, so a near-full high-pop server runs at the max value. Populations not flagged to scale with large maps cap the area at 16 km² (a 4000 map). Refills run every `spawn.tick_populations` seconds, one population per frame, with a per-pass spawn cap.

Why the gain is small. Facepunch, October 2026 devblog: only a subset of NPCs is processed each frame, up to a hard-coded time cap, “so when we increase the population, the server doesn't directly lose FPS, instead NPCs just become slower to react.” Animals far from players also go dormant (`aimanager.ai_dormant` = true, wake range `aimanager.ai_to_player_distance_wakeup_range` = 160 m). Animals number in the hundreds; a mid-wipe high-pop map holds hundreds of thousands of player entities.

What it still buys. Fewer entities to save and network, fewer awake brains around busy areas, and more AI budget per remaining animal. If your values are already near the suggestions below, cutting further gains nothing measurable. After a restart, run `spawn.report` to confirm live counts match the targets.

Convar	2026 default	Suggested high-pop	Convar	2026 default	Suggested high-pop
`bear.population`	2	1	`ridablehorse.population`	2	1
`polarbear.population`	1	0.5	`tiger.population`	0.5	0.25
`wolf.population`	2	1	`panther.population`	0.5	0.25
`boar.population`	5	2.5	`crocodile.population`	1	0.5
`stag.population`	3	1.5	`snakehazard.population`	5	2.5 (optional)
`chicken.population`	3	1.5	`zombie.population`	0	0

Event populations (`halloween.*population`, `halloweendungeon.population`, `xmasdungeon.xmaspopulation`) default to 0 outside events; leave them alone.

2.2 REVERT tickrate Reduced tick rates

`server.tickrate` **is not the simulation tick.** In both the 2023 and 2024 server source it sets one thing: how often `ServerMgr.DoTick()` runs. DoTick processes RCON, the Rust+ companion server, Nexus and authentication time-out kicks. Carbon and Oxide also fire their `OnTick` hook from it. Facepunch's default is 10.

Hosting guides recommending 30, 60 or 128 are repeating the 2016 default, which was 30. Raising it multiplies RCON polling and every plugin's OnTick cost. Lowering it below 10 saves almost nothing and makes RCON and Rust+ less responsive.

Player tick rates are separate replicated convars: `player.tickrate_sv` (default 16) and `player.tickrate_cl` (default 32). Both are clamped in code to 16-128, so anything below 16 is silently raised to 16. Lowering `tickrate_cl` cuts inbound tick packets but coarsens movement and hit validation, and Jobs 3/4 already moved much of that work off the main thread. Use Facepunch's values.

Convar	Default	What it really controls	Recommendation
`server.tickrate`	10	ServerMgr.DoTick: RCON, Rust+, Nexus, auth time-outs, OnTick plugin hook	**10**

Convar	Default	What it really controls	Recommendation
`player.tickrate_sv`	16	Per-player server update rate (ConnectedPlayerUpdate); clamp 16-128	16 (can't go lower)
`player.tickrate_cl`	32	Client tick send rate (replicated to clients); clamp 16-128	32
`ai.tickrate`	5	AI think ticks per second; already under frame budgets	5
`fps.limit`	240	Server frame cap; throughput of every per-frame budget scales with real FPS	240; at least 100 if sharing a host
`spawn.tick_populations`	60 s	Respawn pass interval; time-sliced, one population per frame	Gameplay lever, not perf
`server.planttick`, `server.metabolismtick`, `server.modifiertickrate`	60 / 1 / 1	Gameplay timers (growth, hunger, buffs)	Not perf levers
`decay.tick`	600 s	Minimum gap between decay evaluations per entity	Not a CPU lever (2.3)

2.3 CHECK - MAY HURT Reduced decay rates

What the code does. `ServerBuildingManager.Cycle()` runs every frame. It evaluates **5 building-decay entities and 5 world-decay entities**, round-robin. Each `DecayTick()` returns immediately unless `decay.tick` seconds (default 600) have passed since that entity's last evaluation. The damage applied is proportional to the real elapsed time × `decay.scale`.

- Decay costs a small, fixed amount of CPU per frame whatever you set. Raising `decay.tick` saves nothing measurable, and lowering it doesn't make bases decay faster.
- On a big map the round-robin is the real clock. 300,000 building-decay entities at 50 FPS take about 20 minutes per full pass (300,000 ÷ 5 ÷ 50 = 1,200 s). Total decay is unaffected because damage uses elapsed time.
- `decay.scale` below 1, longer `decay.duration_*` values or added `decay.delay_*` all let abandoned bases live longer. Every surviving block is serialised on every save and snapshotted to every player who walks past. This is the opposite of a performance setting.

Recommendation. `decay.scale 1`, `decay.upkeep true` and default durations. Protect active players through upkeep (TC), not by slowing decay globally. Remove abandoned structures actively (Section 7.5). If you want bases to fall faster, `decay.scale` 1.5-2 is the lever, but that is a gameplay decision.

Decay round-robin constants are from the August 2024 source. Facepunch may have retuned them since, but the per-entity elapsed-time model has been stable for years.

3. Facepunch's 2026 engine work comes first

In 2026 Facepunch has been moving the per-player and per-entity main-thread work onto worker threads. The Spring Clean devblog shows Facepunch profiling a 360-player server and finding AntiHack and network streaming running serially inside a ~20 ms section. Each monthly patch since has attacked that section. For a high-pop server, staying on the newest Jobs level is the single largest performance change available.

When	Update	Server-relevant change	Source
2 Apr	Spring Clean	`server.useplayerupdatejobs` 2 becomes default. Level 3 added as experimental: UniTask instead of C# Tasks (fewer allocations), more AntiHack in Burst jobs, player network streaming parallelised. ~600 MB less runtime memory and ~2.7 GB less disk (reported).	Devblog + patch notes
7 May	Upgrade Hard, Raid Harder	Entity saving made thread-safe (no Unity API calls), the prerequisite for Jobs 4. Memory and network optimisations (reported).	Devblog (secondary)
4 Jun	Built Different	Unity 6. ~20 server allocation sources removed or amortised; improved hit validation. New Facepunch navmesh behind `-useNewNavmesh` (off by default): built in the background after start, then loaded from disk on restart.	Devblog / changelist
2 Jul	Common Ground	Jobs 2 removed, Jobs 3 is the default. Jobs 4 added as experimental.	Changelist
6 Aug	Power Trip	Jobs 4 = parallel “entity saving”: snapshots grouped into batches (ParallelNetworkQueueBatchSize, default 256), each serialised as its own thread-pool task. Synthetic results: 1,000 entities 6.6 -> 3.6 ms; 10,000 entities 64.4 -> 10.6 ms.	Devblog
3 Sep	Breach and Clear	Jobs 4 parallel-save improvements; SyncVar buffer leak during parallel saves fixed; pool leak fixed; server mission memory reduced.	Changelog (secondary)

When	Update	Server-relevant change	Source
1 Oct	Livestock	New navigation system multithreaded, “at least doubling its performance”. IK multithreaded (~1 ms/frame saved at 100 players). Unused map data freed (~175 MB on a 4500 map, reported). New cows and sheep add entities.	Devblog
13 Oct	commit #167854	“Auto adjust jobs thread count when changing CPU affinity.”	Commit log
22-23 Oct	commits	RustRelay TCP packet overhead reduced; websocket-sharp replaced with Fleck (lower websocket/RCON bandwidth overhead).	Commit log
3 Nov	commit #170103	“make UsePlayerUpdateJobs default to 4” - expected live with today's 5 Nov update.	Commit log

What to do with this

- After today's patch, type `server.useplayerupdatejobs` in the console. Expect **4**. If it reads 3, Jobs 4 didn't ship in this build: test 4 on staging for a day before production.
- Delete any `server.useplayerupdatejobs` line from server.cfg and serverauto.cfg so you follow Facepunch's default as it moves. Leave `server.emergencydisableplayerjobs` at true: it runs extra validation and falls back to vanilla processing instead of crashing.
- Don't restrict the process to one or two cores. If you run several instances on one box, give each its own CCD or core group of at least 8 cores and let the auto-sized job threads use them.
- If a plugin misbehaves under Jobs 4, report it to the author (and Facepunch if vanilla). Don't drop the whole server back to an older mode.
- `-useNewNavmesh` is a launch argument, not a convar. Its benefits are a background build, a disk cache (fast restarts) and multithreaded pathing. Test it on staging with your NPC and event plugins first, because anything that samples Unity's NavMesh directly may behave differently.

4. Recommended convar baseline

Defaults below are the current-build values. The recommendation column is a starting point for a 200-300 player PulseRust server. **TEST** means trial on staging first, and “gameplay” marks settings players will notice.

4A. Core loop, threading and saves

Convar	Default	Recommend	Why / risk
`server.useplayerupdatejobs`	3 (4 from 5 Nov)	leave unset	Track Facepunch's default; verify it reads 4.
`server.emergencydisableplayerjobs`	true	true	Safety net that falls back to vanilla instead of crashing.
`fps.limit`	240	240	Use at least 100 on a shared host. Never 30-60 “to save CPU”.
`server.tickrate`	10	10	See 2.2.
`server.saveinterval`	600 s	600 (900)	Use 900 only if the save log shows write+disk over ~0.5 s. Longer means more rollback on a crash.
`server.saveframebudget`	5 ms	5	ms per frame warming save caches before the write.
`server.savebackupcount`	2	2	Rotation is rename-based and cheap; the minimum is 2.
`gc.incremental_milliseconds`	3	3	Incremental GC slice per frame. GC spikes come from plugin allocations; fix those instead.
`global.job_system_threads`	auto	leave unset	Auto-sized; made affinity-aware by a 13 Oct commit.

How saves actually cost you. `SaveRestore.Save()` has three phases. It warms each entity's save cache, spread across frames by `server.saveframebudget`. It writes every cache into one buffer **inside a single frame**. It rotates backups and writes the file **synchronously on the main thread**. The server logs this every save as `Saved N ents, cache(x), write(y), disk(z)`. Chart those numbers across the wipe: write grows with entity count, and disk shows whether storage is too slow (use NVMe).

4B. Entity count and cleanup

Convar	Default	Recommend	Why / risk
`decay.scale`	1	1	Never below 1 for performance (2.3).
`decay.upkeep`	true	true	Upkeep is what lets you keep decay fast without punishing active players.
`server.itemdespawn`	300 s	240 (gameplay)	Dropped items are physics bodies; this also scales loot-bag lifetime.
`server.itemdespawn_quick`	30 s	30	Already short.
`server.corpsedespawn`	300 s	300	Keep. Shorter angers players more than it helps.
`bike.outsidedecayminutes`	1440	480 (gameplay)	Abandoned bikes last 24 h outside by default.
`snowmobile.outsidedecayminutes`	1440	480 (gameplay)	Same as bikes.
`modularcar.outsidedecayminutes`	864	480 (gameplay)	Cars are heavy multi-module physics entities.
`modularcar.population`	3	2 (optional)	Fewer roaming cars to simulate.
`spawn.max_density`	1	1	Also cuts ores and collectibles. Lower it only with clear entity-pressure evidence, or use Entity Reducer for targeted cuts.

The 2026 naval content adds player-built boats with long decay start delays (`playerboat.decaystartdelayminutes` 1440, `tugboat.tugdecaystartdelayminutes` 1440). Watch them in entity reports. They are bases that move.

4C. AI and navmesh

Convar	Default	Recommend	Why / risk
`aimanager.ai_dormant`	true	true	Agents outside player range go dormant.
`aimanager.ai_to_player_distance_wakeup_range`	160 m	160 (140)	Try 140 only if `ai.brainstats` shows most animals awake at peak.
`aithinkmanager.animalframebudgetms`	2 ms	2 (TEST 1.5)	Caps animal AI time per frame; animals react slower.
`aimanager.pathfindingiterationsperframe`	100	100	Facepunch's documented tuning range is 50-500.
`ai.npc_max_junkpile_count`	30	20 (optional)	Fewer roaming junk-pile scientists.
`aimanager.nav_wait`	true	true	Waits for the navmesh before opening, avoiding generation hitches while players are on.
`ai.nav_carve_use_building_optimization`	false	TEST true	Fewer navmesh carve volumes per base (min `ai.nav_carve_min_building_blocks_to_apply_optimization` = 25 blocks).
`aimanager.nav_obstacles_carve_state`	2	TEST 1	1 = only player construction carves the navmesh; 0 = none. NPCs may path through deployables.
`aimanager.nav_disable`	false	false	Never enable on a server with NPCs or animals: they can't move.
`-useNewNavmesh` (launch arg)	off	TEST	See Section 3.

4D. Per-frame budgets (leave at default unless profiling points at them)

These caps already protect frame time. Lower one only when `profile.perfsnapshot` shows that system near its cap in your worst frames, and accept that it will lag. Automation-heavy wipes are the usual reason:

Convar	Default	Recommend	Note
`server.industrialtransferstricttimelimits`	true	true	Aborts slow transfers (uneven splitting) to protect frame time.
`server.maxitemstacksmovedpertickindustrial`	12	12 (TEST 8)	Stacks one conveyor moves per tick.
`server.industrialframebudgetms`	0.5 ms	0.5	Industrial job queue.
`server.hopperanimationbudgetms`	0.1 ms	0.1	0 or less = items move instantly (no animation work).
`autoturret.scan_budget_ms`	0.5 ms	0.5	Renamed from auto_turret_budget_ms.
`growableentity.framebudgetms`	0.25 ms	0.25	Large farms on high pop can saturate this.
`ioentity.framebudget*`	0.25-1 ms	default	Lower the low-priority electric budget first if needed.

4E. Physics, network and AntiHack

Convar	Default	Recommend	Why / risk
`physics.droppedmode`	2	TEST 3	Unity collision mode for dropped items and corpses: 2 = ContinuousDynamic, 3 = ContinuousSpeculative (cheaper; already used for server ragdolls). Watch for items clipping through floors.
`physics.sendeffects`	true	true	Collision effects to clients; negligible saving.
`server.encryption`	2	2	Don't trade security for CPU; network work is threaded.
`server.netcache`	true	true	Caches entity network serialisation.
`antihack.*`	defaults	defaults	Jobs 3 moved AntiHack work into Burst jobs. Disabling protections trades cheaters for microseconds.

5. What high-pop servers do: hardware and operations

Large networks don't publish their configs, and no current settings from the big high-pop operators turned up in public sources. What is documented, and consistent across admin forums, Facepunch's own notes and hosting guides:

Area	Practice	Evidence
CPU	Single-core speed still sets the ceiling (main thread). Since 2026, worker threads matter too: Jobs 3/4, navigation and IK. Give each high-pop instance at least 8 fast cores, ideally a whole CCD, and never share a CCD between two busy instances. X3D parts are popular, but no Rust-server benchmark was found; measure before buying.	Facepunch devblogs; admin consensus
RAM	32 GB or more per high-pop instance with plugins. Memory grows across a wipe; the Livestock update freed ~175 MB of map data.	Hosting guidance; Livestock notes
Storage	NVMe. The save's write and disk phases run synchronously on the main thread.	Server source
OS	No measurable Linux vs Windows difference in community reports. Run what your team operates best.	uMod / Oxide forums, r/playrustadmin
Restarts	Daily off-peak restart with a countdown (Smooth Restarter, Timed Execute). Admins report memory creep over multi-day uptime. Restart before peak, never during it.	Oxide forum threads
Wipe / map	Entity count sets the practical wipe length. Rustoria's 2018 post described wiping because ~175k entities broke plugins and restarts; builds handle far more now, but the curve is the same. 3500-4500 maps for 150-300 pop; custom-map prefab count adds fixed entities.	Rustoria Steam post; hosting guides
Framework	Carbon is the common choice on busy modded servers for its tooling and update speed. No independent Carbon vs Oxide benchmark exists, so plugin quality dominates either way.	Carbon repo; hosting comparisons
Staging	A staging server that mirrors plugins and config. Test each wipe's patch, Jobs level and plugin updates there first.	Standard practice

6. Myths and dead convars to delete

Each line below was checked against the 2023/2024 source and against both 2026 command lists. Delete these from server.cfg, serverauto.cfg and any launch scripts:

Setting	Reality	Action
`gc.buffer 4096`	Client-only ([ClientVar], no [ServerVar]) since at least 2019. The buffer logic lives in Rust.GC, an IClientComponent, so the server never runs it.	Delete
`gc.interval`	Not present in 2023+ builds.	Delete
`physics.steps`, `physics.minsteps`	Existed in 2023; absent from both 2026 command lists.	Delete
`global.asyncwarmup`, `global.preloadconcurrency`, `global.warmupconcurrency`	Removed.	Delete
`batching.colliders`, bare `nav_disable`	Don't exist. The navmesh switch is `aimanager.nav_disable` and must stay false.	Delete
`autoturret.auto_turret_budget_ms`	Renamed to `autoturret.scan_budget_ms`.	Rename / delete
`sleepers.on`, `truth.enforce`, `cheaters.log`, `server.maxticketspersecond`, `server.clienttimeout`, `server.globalchat`	Old or Legacy-era convars; absent from current builds, so they do nothing.	Delete

Setting	Reality	Action
`server.useplayerupdatejobs 0/1/2`	Modes 0/1 deprecated; mode 2 removed in July 2026.	Delete
`server.tickrate 30/60/128`	Not the simulation tick (2.2).	Set 10
`fps.limit 30/60` "to save CPU"	Throttles every per-frame budget (Section 1).	Set 240
Inline comments, e.g. `fps.limit 240 # cap`	Rust only skips lines whose first character is `#`. Trailing text is passed to the command.	Own-line comments only

Load order matters: `server.readcfg` *runs* `serverauto.cfg` *first and* `server.cfg` *second, so server.cfg wins.* `server.writecfg` *rewrites serverauto.cfg with every saved convar (fps.limit is one), so check it for stale values after any admin runs it.*

7. Plugin performance

7.1 Framework

PulseRust already runs Carbon; the Kits/PulseDailyGems audit cites Carbon profiler output. Keep it. Carbon's repository is actively maintained (last commit on 5 November 2026) and has the most complete built-in profiler in the ecosystem. Update Carbon in the same maintenance window as the Rust patch every wipe.

7.2 Profiling toolkit (all verified in source)

Tool	Command	What it tells you
Carbon	`c.profile 60` then `c.profiler.print -t`	Native Mono profile of everything for 60 s (flags -cm -am -t -c -gc). Exports as table, CSV, JSON or ProtoBuf.
Carbon	`c.profile.plugin Kits 60`	Recompiles one plugin with profiler hooks, records, then reverts. Needs "TrackCalls": true in config.profiler.json.
Carbon	`c.hooks`, `c.hookinfo OnEntitySpawned`	Live hooks; per-hook time, memory and lag-spike counts, and which plugins subscribe.
Oxide (reference)	`o.plugins`	Per-plugin cumulative hook time and hook-allocated memory. Oxide logs any single hook call over 100 ms, tagged [GARBAGE COLLECT] when a GC ran during it.
Facepunch	`profile.perfsnapshot 15 Peak 10`	Writes a JSON perf snapshot of up to 10 frames (arguments: delay, name, frames), viewable in Perfetto. Shows main and worker threads.
Facepunch	`profile.watchallocs`	Tracks allocations and dumps a .json.gz with stacks once conditions are met.
Facepunch	`profile.profiling_lagspikes` + `profile.rpc_lagspike_threshold`	Records inbound RPCs and console commands that caused lag spikes (RCON has its own threshold).
RCON	`serverinfo`	EntityCount, Framerate, Memory, Collections (GC count), NetworkIn/Out. Log it every minute.
Console	`spawn.report`, `ai.brainstats`, `pool.print_memory`, `physics.print_colliders_per_prefab`	Population fill vs target, live AI brains, object pools, and the prefabs carrying the most colliders.

If `c.profile` *reports that the Mono profiler couldn't start, check the server log. Carbon also publishes a dedicated profiler build on its GitHub releases page.*

7.3 The hooks that dominate on high pop

Hook	How often it fires	Rule for plugin code
`CanNetworkTo`	Per entity, per client, on every network send	Avoid unless essential (vanish/hide). Exit on the first cheap check.
`OnPlayerInput`, `OnPlayerTick`	Every player tick (16-32 Hz × players)	No allocations, LINQ, string building or permission lookups; cache per player.
`OnEntitySpawned`, `OnEntityKill`	Every entity, including the whole world at startup	Type-check first, then defer real work to a queue.
`OnEntityTakeDamage`	Every hit; explosive raids fire thousands	Cheapest rejections first; no Physics queries.
`OnItemAddedToContainer`, `CanAcceptItem`	Every item move; conveyors and crafting	Very hot on automation wipes; keep to O(1) work.
`OnTick`	server.tickrate per second	Prefer a timer at the cadence you need.

Hook	How often it fires	Rule for plugin code
`OnServerSave`	Every saveinterval	Never write files synchronously here; add 5-25 s random jitter and write on a worker thread.

7.4 Patterns to remove (and where PulseRust already has them)

- **Synchronous disk I/O on the main thread**: File.ReadAllText/WriteAllText or JSON serialise on player actions, connects, disconnects or saves. Found in Kits (K-01), PulseDailyGems (DG-01), UltimateLeaderboard (UL-02, UL-03) and ClanTree (CT-01..03). It compounds with Rust's own single-frame save write. Fix with the Kits v2 PlayerSaveWorker pattern: dirty flag, background thread, binary format and random save jitter.
- **Whole-world scans on timers**: iterating `BaseNetworkable.serverEntities` or calling FindObjectsOfType every few seconds. Track the entities you care about in spawn/kill hooks instead, or time-slice the scan with a coroutine as Entity Cleanup does.
- **Per-second UI rebuilds for every player**: HUD and info panels that re-send full CUI JSON. Send once, then update only the changed element, and pre-serialise static layouts to byte[] (as Kits v2 and UltimateLeaderboard already do).
- **Allocation-heavy hot hooks**: LINQ, string concatenation, boxing and new lists inside the hooks in 7.3. They don't show in hook time; they show up as GC collections and [GARBAGE COLLECT] hitches.
- **Large physics queries**: OverlapSphere or Vis.Entities with big radii in frequent hooks or timers.

7.5 Performance-oriented plugins (confirmed on uMod)

Plugin (author)	Use	Caveat
Entity Cleanup (2CHEVSKII)	Removes entities outside building privilege or not authorised on the TC; time-sliced coroutine.	Run off-peak; review its config before the first run.
Auto Decay	Applies decay damage to objects outside building zones.	Overlaps vanilla decay; configure deliberately.
Entity Limit (Orange / The Friendly Chap)	Caps entities per player or per building.	Gameplay rule, but the most direct brake on late-wipe growth.
Entity Reducer (Arainrr)	Sets exact target counts for any spawn population: ores, collectibles, barrels, animals.	Disables population scaling for populations it manages.
Despawn Config (Wulf)	Per-item and container despawn times.	Finer-grained than `server.itemdespawn`.
Performance Monitor (Orange)	Collects server performance information.	Complements the Carbon profiler.
Smooth Restarter / Timed Execute	Scheduled restarts with countdowns.	-

Every plugin is itself a cost. Prefer a vanilla convar where one does the job, and profile new plugins on staging under bots or load before production.

7.6 PulseRust plugin backlog from earlier audits

These were the Critical findings in the four previous reports. If any are still open, fix them before tuning convars: they produce the 100 ms+ stalls that no server setting can absorb.

ID	Plugin	Finding
K-01	Kits	Synchronous disk save on every kit redemption
K-02	Kits	GetOrCreateKitData() silently creates player records on every UI panel build
DG-01	PulseDailyGems	Synchronous disk save on every gem claim
UL-01	UltimateLeaderboard	GetEntry() blocks on stat-event miss
UL-02	UltimateLeaderboard	LoadAllClans() synchronous on startup
UL-03	UltimateLeaderboard	60 serializations on main thread per save
CT-01	ClanTree	Synchronous File.ReadAllText on player connect
CT-02	ClanTree	Coroutine HandleSaves() still writes JSON on main thread
CT-03	ClanTree	SaveClanData() called synchronously on player disconnect

8. Measurement and rollout plan

Baseline first (one peak day, before changing anything)

- Log `serverinfo` every 60 s: Framerate, EntityCount, Memory and Collections. Chart FPS against entity count across the wipe.
- Pull every `Saved N ents, cache(x), write(y), disk(z)` line from the server log.
- At peak, capture `profile.perfsnapshot 15 PeakDay1 10` and a Carbon `c.profile 120`.
- Record `spawn.report` and `ai.brainstats` once at peak.

Change order (one group at a time; 48 h between groups)

Step	Changes	Player-visible?
1	Jobs level check, CPU cores/affinity, fps.limit 240, server.tickrate 10	No
2	Delete dead convars; clean serverauto.cfg	No
3	Plugin fixes from 7.6; stagger OnServerSave	No
4	Entity hygiene: decay.scale 1, cleanup plugin, vehicle/item lifetimes. Announce to players first.	Yes
5	TEST items on staging, one at a time: -useNewNavmesh, carve options, physics.droppedmode 3, lower budgets	Possibly

Success looks like

- Peak FPS stays flat from wipe day to the end of the wipe instead of sliding with entity count.
- Save write+disk stays under ~0.5 s at end of wipe.
- No hook-time warnings or lag-spike records attributed to PulseRust plugins.
- GC collections per minute stay flat at peak.

Appendix A - server.cfg snippet

Also committed as `PulseRust_HighPop_Recommended.cfg`. Merge it into `server/<identity>/cfg/server.cfg`; don't paste it over the top. Lines starting with `#` are ignored by Rust.

```
# ========================================================================
# PulseRust high-pop performance baseline - researched 5 Nov 2026
# See PulseRust_HighPop_ConvarGuide.pdf for the reasoning behind each line.
# Merge into server/<identity>/cfg/server.cfg (server.cfg loads AFTER
# serverauto.cfg, so values here win). Rust skips a line only when its
# FIRST character is '#': never put a comment after a value.
# ========================================================================
#
# --- Threading ----------------------------------------------------
# Leave server.useplayerupdatejobs UNSET so you follow Facepunch's
# default (4 from the 5 Nov 2026 patch, per commit #170103). Check it in
# the console after patching; if it still reads 3, test 4 on staging.
server.emergencydisableplayerjobs true
#
# --- Frame cap and tick (defaults; overrides "performance" myths) ----
fps.limit 240
server.tickrate 10
#
# --- Saves --------------------------------------------------------
# Raise to 900 only if the log's "Saved N ents ... write(y), disk(z)"
# shows write+disk above ~0.5 s late in the wipe.
server.saveinterval 600
server.saveframebudget 5
#
# --- Decay: never slow decay for performance ----------------------
decay.scale 1
decay.upkeep true
#
# --- Animals: ~50% of 2026 defaults (AI is already frame-budgeted) --
bear.population 1
polarbear.population 0.5
```

```
wolf.population 1
boar.population 2.5
stag.population 1.5
chicken.population 1.5
ridablehorse.population 1
tiger.population 0.25
panther.population 0.25
crocodile.population 0.5
#
# --- Dropped items and abandoned vehicles (gameplay trade-off) ------
server.itemdespawn 240
bike.outsidedecayminutes 480
snowmobile.outsidedecayminutes 480
modularcar.outsidedecayminutes 480
modularcar.population 2
#
# --- AI --------------------------------------------------------------
aimanager.ai_dormant true
ai.npc_max_junkpile_count 20
#
# --- TEST ON STAGING FIRST: remove the leading '#' one line at a time -
#physics.droppedmode 3
#ai.nav_carve_use_building_optimization true
#aimanager.nav_obstacles_carve_state 1
#aithinkmanager.animalframebudgetms 1.5
#server.maxitemstacksmovedpertickindustrial 8
# Launch argument (not a convar) to trial on staging: -useNewNavmesh
#
# --- DELETE from server.cfg / serverauto.cfg / launch scripts if present
# gc.buffer, gc.interval, physics.steps, physics.minsteps,
# global.asyncwarmup, global.preloadconcurrency, global.warmupconcurrency,
# batching.colliders, nav_disable, autoturret.auto_turret_budget_ms,
# sleepers.on, truth.enforce, cheaters.log, server.maxticketspersecond,
# server.clienttimeout, server.globalchat, server.useplayerupdatejobs 0/1/2
```

Appendix B - Sources

Server source, convar lists and framework code (primary, read directly)

- MillionthOdin16/RustChangelog: decompiled server, Aug 2024: ServerMgr, ServerBuildingManager, DecayEntity, SaveRestore, SpawnHandler, DensitySpawnPopulation, ConVar/Server, ConVar/Player, ConVar/Physics, ConVar/Admin
- Ailtop/RustDocuments: decompiled server, late 2023: ConVar/GC, ConVar/FPS, ConsoleCommands.json
- unet-dev/Decompiled-Assemblies: 2019: ConVar/GC, Facepunch.Console ConsoleSystem.RunFile (comment handling)
- Klobiroff/Rust-217Devblog-Source: Rust/GC.cs (client-only GC buffer component)
- dexsper/RustRcon: 1,651 commands with help text generated from the game's ConsoleGen (commit 21 Apr 2026)
- Crushedice/RustManagerAgent: 2025-26 convar value dump (knowledge/verified/variables_import_lines.jsonl)
- CarbonCommunity/Carbon: Core.Commands.Profiler.cs, Core.Commands.Hooks.cs
- OxideMod/Oxide.Core / Oxide.Rust: Plugin.cs hook timing and warnings; RustCommands.cs; Rust.opj hook manifest
- publicrust/umod-pluigns-dataset: uMod plugin snapshot used to confirm plugin names and behaviour

Facepunch 2026 devblogs and changelogs (read via search extracts; site not reachable from the research environment)

- https://rust.facepunch.com/news/spring-clean: Jobs 2 default, Jobs 3 experimental, 360-player profile
- https://rust.facepunch.com/news/upgrade-hard-raid-harder: thread-safe entity saving
- https://rust.facepunch.com/news/built-different: Unity 6, allocation work, -useNewNavmesh
- https://rust.facepunch.com/news/common-ground, https://rust.facepunch.com/changelist/4043: Jobs 3 default, Jobs 2 removed
- https://rust.facepunch.com/news/power-trip: Jobs 4 parallel entity saving, benchmark figures
- https://rust.facepunch.com/news/livestock: multithreaded navigation and IK, NPC frame cap quote
- https://rust.facepunch.com/changes: changelog entries (parallel save leak fix, mode changes)
- https://rustfront.com/news/upcoming-2026-11: commit log preview: #167854, #168889, Fleck swap, #170103
- https://mixed-news.com/en/rust-npc-per-frame-budget-facepunch-multithreaded-navmesh/: coverage of the Livestock NPC budget and navmesh claims
- https://pinehosting.com/blog/rust-september-update-2026-breach-and-clear-monument-blockers-hqm-nodes-and-more/: September Jobs 4 improvements (secondary)

Community and operations evidence (lower confidence)

- https://steamcommunity.com/groups/rustoria/discussions/0/1620600279666568817: Rustoria 2018: entity-driven wipes, ~175k entity breaking point
- https://oxidemod.org/threads/rust-server-eating-more-ram-over-time.11698: RAM growth and daily restarts
- https://umod.org/community/rust/11080-better-server-platform-linux-or-windows: Linux vs Windows discussion
- https://physgun.com/help/game-hosting/rust/how-map-size-affects-rust-server-performance/: map size vs population guidance
- https://eastgate.host/rust-server-optimization-high-entities-low-fps/, https://xgamingserver.com/blog/how-do-we-improve-rust-server-performance/, https://supercraft.host/wiki/rust/performance_optimization/: examples of hosting-guide advice this report corrects (tickrate 30-60, fps caps, gc.buffer variants)